

Ma Place — ajouter les files et les tickets

Explication de la nouvelle modélisation et guide détaillé des essais manuels Postman réalisés — 2 octobre 2026

Objectif de ce document

Ce compte-rendu explique ce qui a été ajouté dans le backend, pourquoi une entité queue (file d'attente) est nécessaire entre un établissement et ses tickets, comment les données sont reliées, et comment rejouer les requêtes Postman avec des exemples.

1. Pourquoi ajouter une table queues ?

Dans la première modélisation, un ticket était rattaché directement à un établissement. Cela identifie le lieu, mais ne dit pas à quelle session de service appartient le ticket. Si le même établissement ferme sa file puis la rouvre, l'application doit pouvoir distinguer les tickets de l'ancienne file de ceux de la nouvelle.

La table queues représente cette session. Le schéma la décrit comme une file de la journée et l'application conserve les files fermées. Lorsqu'une file est fermée, les tickets passés restent attachés à cette ancienne file ; quand l'établissement ouvre à nouveau, une nouvelle ligne queues est créée et sa numérotation recommence à zéro. Le code actuel crée donc une nouvelle file à chaque réouverture après fermeture ; il n'impose pas une contrainte calendaire « une seule file par date ».

Sans queues, il serait difficile de conserver proprement l'historique, de remettre le compteur à zéro sans confondre les anciens tickets, et de savoir si un ticket est dans la file en cours ou dans une file terminée.

Avant / après

Avant : establishments —< tickets
Après : establishments —< queues —< tickets

La relation après changement est : un établissement peut avoir plusieurs files successives ; une file appartient à un établissement ; une file contient plusieurs tickets ; un ticket appartient à une seule file.

2. Décortiquage des tables ajoutées

Table / colonne	Rôle expliqué
queues.id	Identifiant UUID propre à une session de file. C'est l'identifiant de cette file, pas celui de l'établissement ni du ticket.
queues.establishment_id	Clé étrangère vers establishments.id : rattache la file à son propriétaire. ON DELETE CASCADE supprime les files si l'établissement est supprimé.
queues.status	État de la file : OPEN (accepte des tickets et appels), PAUSED (conserve la file mais bloque ces opérations), CLOSED (session terminée).
queues.last_number	Dernier numéro déjà attribué. Il débute à 0 ; à chaque nouveau ticket accepté, il augmente de 1. Une contrainte empêche les valeurs négatives.
queues.opened_at / closed_at	Dates d'ouverture et de fermeture. closed_at reste NULL tant que la file est ouverte ou en pause ; il est renseigné à la fermeture. Une contrainte vérifie cette cohérence.
tickets.queue_id	Remplace le rattachement direct au seul établissement : indique exactement la session de file à laquelle le ticket appartient. Référence queues.id.
tickets.number	Numéro visible par le visiteur, unique dans une file grâce à UNIQUE(queue_id, number). Le même numéro peut exister dans une autre file/journée.
tickets.status	État du visiteur : WAITING, SERVING, COMPLETED ou CANCELLED. Les statuts ticket et file sont distincts car ils décrivent deux choses différentes.

Préservation de l'intégrité : la création d'un ticket se fait dans une transaction : le modèle verrouille la file, augmente last_number et insère le ticket. Si l'opération échoue, la transaction est annulée. L'indexation sur les IDs, statuts et numéros facilite les recherches.

queue_status sur establishments : il reste un miroir pratique du statut courant utilisé par la liste publique. La source détaillée et l'historique sont dans queues.status ; le champ miroir est mis à jour en même temps que les transitions.

3. Préparer Postman et l'authentification

Démarrer le serveur backend (depuis le dossier backend) avec `npm run dev`. La base PostgreSQL doit être démarrée et configurée. URL locale utilisée : `http://localhost:3000`.

Repères Postman

Pour les routes protégées, ouvrir l'onglet Authorization, choisir Bearer Token et coller seulement le token JWT. Postman ajoute lui-même le préfixe Bearer. Ne pas coller le token dans l'URL et ne pas le publier.

Le script seed du projet contient un compte de démonstration Salon Éléance : `salon@example.com / password123`. Ce sont des identifiants de développement uniquement ; ne pas les utiliser en production. On peut aussi se connecter avec tout compte valide existant dans la base.

Test 1 — Connexion : récupérer le token

POST `http://localhost:3000/api/auth/login`

Authorization : Aucun (route publique)

Body → raw → JSON :

```
{
  "email": "salon@example.com",
  "password": "password123"
}
```

Réponse attendue : 200 OK ; la réponse contient `data.token` et `data.establishment.id`. Utiliser l'ID de l'établissement dans la requête de création de ticket.

```
{
  "status": "success",
  "message": "Connexion réussie.",
  "data": { "token": "<JWT_A_GARDER_SECRET>", "establishment": { "id": "<UUID_ETABLISSEMENT>", "name": "Salon Éléance" } }
}
```

Test 2 — Profil lié au token

GET `http://localhost:3000/api/auth/me`

Authorization : Bearer Token obtenu à la connexion

Réponse attendue : 200 OK ; retourne l'établissement associé au JWT.

Test complémentaire non confirmé dans notre session — santé API

GET `http://localhost:3000/health`

Authorization : Aucun

Réponse attendue : Réponse prévue par le code : status ok, database connected et timestamp. Cette requête n'a pas été rapportée comme testée dans Postman.

4. Tester le cycle de vie de la file

Toutes les routes `/api/queue` exigent le même Bearer Token. Elles utilisent l'ID du compte porté par le token : aucun `établissement_id` n'est envoyé dans le body.

Test 3 — Ouvrir une file

POST `http://localhost:3000/api/queue/open`

Authorization : Bearer Token

Réponse attendue : 200 OK ; `data.queue.status` vaut OPEN, `last_number` vaut 0 et `closed_at` vaut null.

Test 4 — Consulter la file et ses tickets

GET `http://localhost:3000/api/queue`

Authorization : Bearer Token

Réponse attendue : 200 OK ; retourne `queue`, `queueStatus` et `tickets` de la file active.

Test 5 — Mettre la file en pause

POST `http://localhost:3000/api/queue/pause`

Authorization : Bearer Token

Réponse attendue : 200 OK ; `data.queue.status` vaut PAUSED. Pendant la pause, les créations de tickets et appels sont refusés.

Test 6 — Reprendre la file

POST `http://localhost:3000/api/queue/resume`

Authorization : Bearer Token

Réponse attendue : 200 OK ; le statut revient à OPEN sans créer une nouvelle file et sans remettre `last_number` à zéro.

Test 7 — Fermer la file

POST `http://localhost:3000/api/queue/close`

Authorization : Bearer Token

Réponse attendue : 200 OK ; le statut devient CLOSED et `closed_at` reçoit la date de fermeture. Une réouverture ultérieure crée une nouvelle file.

5. Tester le parcours d'un ticket

Les exemples suivants utilisent les variables de remplacement `{{establishmentId}}` et `{{ticketId}}`. Dans Postman, on peut les remplacer par les UUID reçus dans les réponses précédentes, ou créer des variables dans un environnement.

Test 8 — Ajouter un ticket visiteur

POST `http://localhost:3000/api/tickets`

Authorization : Aucun (route publique)

Body → raw → JSON :

```
{
  "establishmentId": "{{establishmentId}}",
  "name": "Test Postman",
  "phone": "0600000000"
}
```

Réponse attendue : 201 Created ; ticket avec un id, un `queue_id`, un `number` et le statut `WAITING`. Cette route n'accepte le ticket que si la file est `OPEN`.

```
{
  "status": "success",
  "data": { "ticket": { "id": "{{ticketId}}", "queue_id": "<UUID_FILE_ACTIVE>", "number": 1, "name": "Test Postman", "status": "WAITING" } }
}
```

À retenir : `establishmentId` est l'ID de l'établissement, et non le `queue_id`. Le backend recherche la file ouverte associée, incrémente `last_number` puis relie le nouveau ticket à cette file.

Test 9 — Appeler le prochain visiteur

POST `http://localhost:3000/api/queue/next`

Authorization : Bearer Token

Réponse attendue : 200 OK ; le plus petit numéro en `WAITING` passe à `SERVING`. Si un ticket était déjà `SERVING`, le modèle le marque `COMPLETED` avant de passer le suivant en `SERVING`.

Test 10 — Consulter un ticket publiquement

GET `http://localhost:3000/api/tickets/{{ticketId}}`

Authorization : Aucun

Réponse attendue : 200 OK ; renvoie le statut, la file et l'établissement. Si le ticket est `WAITING`, fournit `position` et `peopleAhead` ; sinon `position` est `null`.

Test 11 — Terminer le service

POST `http://localhost:3000/api/tickets/{{ticketId}}/complete`

Authorization : Bearer Token

Réponse attendue : 200 OK ; autorisé pour le propriétaire de la file si le ticket est `SERVING` ; le statut devient `COMPLETED`.

`COMPLETED` signifie que le client a été servi. Ce n'est ni la fermeture de la file ni une annulation. Le ticket reste consultable et conserve son historique.

Test 12 — Tester une annulation (sur un autre ticket WAITING)

POST `http://localhost:3000/api/tickets/{{ticketId}}/cancel`

Authorization : Bearer Token

Réponse attendue : 200 OK ; seul un ticket `WAITING` de cet établissement peut être annulé ; son statut devient `CANCELLED`.

Pour tester l'annulation, créer d'abord un second ticket dans une file `OPEN` et l'annuler avant `/queue/next`. Un ticket déjà `SERVING`, `COMPLETED` ou appartenant à un autre établissement ne doit pas être annulé via ce parcours.

Exemple de consultation de ticket terminé

```
{
  "status": "success",
  "data": { "ticket": {
    "id": "{{ticketId}}", "queueId": "<UUID_FILE>", "number": 1,
    "status": "COMPLETED", "position": null, "peopleAhead": 0,
    "queueStatus": "CLOSED",
    "establishment": { "id": "{{establishmentId}}", "name": "Salon Élégance" }
  } }
}
```

Un ticket peut donc rester COMPLETED alors que sa queueStatus est CLOSED : l'un est le résultat de service du ticket, l'autre est l'état global de sa session de file.

6. Consultation des établissements, résultats et correction

Test 13 — Lister les établissements

GET `http://localhost:3000/api/establishments`

Authorization : Aucun (route publique)

Réponse attendue : 200 OK ; liste triée par nom, avec id, name, email, phone et queue_status. Le hash du mot de passe ne doit jamais être exposé.

Test 14 — Lire un établissement par ID

GET `http://localhost:3000/api/establishments/{{establishmentId}}`

Authorization : Aucun (route publique)

Réponse attendue : 200 OK ; détail du compte sélectionné.

Résultats rapportés dans Postman

Connexion et `/auth/me` : réussis. File : ouverture OPEN, pause PAUSED, reprise OPEN puis fermeture CLOSED réussies. Ticket principal : création WAITING, appel SERVING, fin COMPLETED et consultation réussis. Second ticket : création WAITING puis annulation CANCELLED réussies. Liste et détail publics des établissements réussis.

Une fermeture de file a d'abord renvoyé 500. Le terminal a indiqué PostgreSQL 42P08 (« types incohérents déduits pour le paramètre \$2 », text versus queue_status). Le paramètre était utilisé à la fois dans l'affectation d'un enum et dans la condition CASE. Un cast explicite vers queue_status a été ajouté dans `src/modele/queueModel.js` ; après cette correction, le test Postman de fermeture a réussi et `closed_at` était renseigné.

Résultats à ne pas confondre

Les vérifications ci-dessus sont des essais manuels de routes et du parcours nominal, pas une couverture complète des erreurs. GET / et GET `/health` n'ont pas été confirmés dans la session. On peut ensuite tester : token manquant/invalid (401), UUID invalide (400), ressource inexistante (404), transition interdite (409), création/appel alors que la file est PAUSED ou CLOSED (409), aucun ticket en attente (404), et second appel à une fermeture déjà faite (409).

Sécurité : un token JWT a été partagé dans la conversation. Par précaution, renouveler JWT_SECRET puis se reconnecter ; éviter de mettre des tokens réels dans un document partagé. Les UUID illustratifs ci-dessus sont des variables à remplacer par les valeurs de son environnement.

Fichiers du projet à consulter

`backend/database/schema.sql` — schéma, enums, relations, contraintes et index. `backend/src/modele/queueModel.js` — transactions de file. `backend/src/modele/ticketModel.js` — attribution et suivi de ticket. `backend/src/routes/` et `src/controllers/` — endpoints et réponses HTTP.

Annexe — tâches réalisées

Cette annexe récapitule les travaux visibles dans le dépôt et les validations manuelles effectuées dans Postman.

A. Changements réalisés dans le backend

Domaine	Tâches réalisées
Modèle de données	Ajout de la table queues entre établissements et tickets ; définition des états OPEN / PAUSED / CLOSED ; ajout de last_number et des dates d'ouverture/fermeture. Remplacement du rattachement direct du ticket à l'établissement par tickets.queue_id. Ajout des contraintes, clés étrangères, unicité du numéro par file, index et triggers updated_at.
Modèles métier	Création de queueModel et ticketModel : ouverture et transitions de file, sélection de la file courante, attribution transactionnelle des numéros, création, appel, consultation, fin de service et annulation des tickets. Les règles vérifient les statuts et le propriétaire du ticket.
Services et contrôleurs	Ajout des services et contrôleurs queue et ticket pour appliquer les règles métier et produire les réponses JSON avec les statuts HTTP adaptés.
Routes et branchement Express	Ajout de /api/queue et /api/tickets dans l'application. Les opérations de gestion de file et les actions établissement sur ticket utilisent le middleware JWT ; la création et la consultation publique d'un ticket restent accessibles sans token.
Données de démonstration	Adaptation du seed pour créer les établissements, leurs files puis les tickets associés ; les mots de passe seedés sont hachés avec bcrypt. Le seed efface d'abord les tickets et établissements : à lancer uniquement sur une base de test.
Configuration / contrôle	Ajout du modèle de variables dans backend/.env.example et du script check-schema.mjs pour inspecter tables, enums, colonnes, index, contraintes et triggers PostgreSQL.
Correction SQL	Après le 500 de /api/queue/close, ajout d'un cast explicite de \$2 vers queue_status dans la transition SQL, pour que PostgreSQL interprète correctement le paramètre enum. La fermeture a ensuite été validée dans Postman.

En résumé : le travail ne se limite pas à deux tables ; il relie schéma, logique transactionnelle, API, authentification, données de démonstration et vérification Postman.

B. Tests API réalisés manuellement dans Postman

Test	Résultat observé
POST /api/auth/login puis GET /api/auth/me	Connexion de Salon Élégance réussie ; le Bearer Token permet de récupérer le compte connecté.
POST /api/queue/open → pause → resume → close	Ouverture OPEN, pause PAUSED, reprise OPEN puis fermeture CLOSED avec closed_at renseigné. La première fermeture a échoué avant la correction du cast SQL.
POST /api/tickets puis POST /api/queue/next	Ticket créé en WAITING avec le numéro 1 de la file active, puis passé en SERVING.
POST /api/tickets/:id/complete et GET /api/tickets/:id	Ticket terminé en COMPLETED et consultable ; les détails montraient son appartenance à l'ancienne file fermée.
POST /api/tickets/:id/cancel	Second ticket créé en WAITING, puis annulé avec succès : statut CANCELLED.
GET /api/establishments et GET /api/establishments/:id	Liste et détail publics réussis. Les réponses ne contenaient pas de hash de mot de passe.

À distinguer : ce sont des essais manuels Postman, pas une suite automatisée exhaustive. Le script npm test est encore un placeholder (« no test specified »). Les cas d'erreur et GET /, GET /health restent à vérifier.

Fichiers principaux : database/schema.sql, database/seed.js, app.js, src/modele/queueModel.js, src/modele/ticketModel.js, src/services/queueService.js, src/services/ticketService.js, src/controllers/queueController.js, src/controllers/ticketController.js, src/routes/queueRoutes.js, src/routes/ticketRoutes.js et check-schema.mjs.